

UTS
PENGOLAHAN CITRA



NAMA : Muhammad Nafis Dhiyaulhaq

NIM : 202431161

KELAS : F

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC :

ASISTEN : 1. Erin Katholica Sakrally Putri Marrison
2. Morgan Immanuel Nainggolan

INSTITUT TEKNOLOGI PLN
TEKNIK INFORMATIKA
2025/2026

DAFTAR ISI

BAB I	3
PENDAHULUAN	3
1.1 Rumusan Masalah	3
1.2 Tujuan Masalah	3
1.3 Manfaat Masalah	3
BAB II	4
LANDASAN TEORI	4
BAB III	8
HASIL	8
BAB IV	19
PENUTUP	19
DAFTAR PUSTAKA	20

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

- Bagaimana cara mengimplementasikan teknik segmentasi dan ekstraksi objek berdasarkan ruang warna HSV (Hue, Saturation, Value) menggunakan pustaka OpenCV pada Python?
- Bagaimana cara membaca dan menganalisis distribusi intensitas cahaya serta karakteristik warna pada suatu citra melalui visualisasi grafik histogram?
- Bagaimana pengaruh penerapan transformasi linear terhadap manipulasi piksel untuk meningkatkan kecerahan (*brightness*) dan kontras pada citra digital?

1.2 Tujuan Masalah

- Mampu melakukan konversi ruang warna BGR ke HSV dan menerapkan teknik thresholding untuk mengekstraksi objek berwarna secara spesifik (biru, hijau, dan merah).
- Mahasiswa mampu membangun dan mengevaluasi visualisasi histogram RGB untuk mengidentifikasi distribusi bimodal serta tingkat pencahayaan citra.
- berhasil mengimplementasikan parameter alpha dan beta pada fungsi transformasi linear untuk mengatur tingkat kecerahan dan ketajaman kontras sebuah citra hitam-putih (grayscale).

1.3 Manfaat Masalah

Kegiatan ini bertujuan untuk memperkuat pemahaman konseptual mengenai citra digital sebagai representasi matriks multidimensi, termasuk pemahaman tentang model ruang warna serta prinsip matematis yang mendasari operasi piksel dan pembentukan histogram. Selain itu, proses pembelajaran juga difokuskan pada peningkatan kemampuan dalam memanfaatkan library Python yang umum digunakan dalam bidang Computer Vision, seperti OpenCV, NumPy, dan Matplotlib, untuk menyelesaikan berbagai kasus dasar pengolahan citra digital. Di samping aspek teknis, kegiatan ini turut mengembangkan kemampuan analisis visual terhadap kualitas gambar sehingga pengguna dapat menentukan metode manipulasi parameter yang sesuai, seperti pengaturan brightness dan contrast, guna memperbaiki detail citra dan menghasilkan tampilan visual yang lebih optimal.

BAB II

LANDASAN TEORI

A. Pemanfaatan Library Python

Dalam pengolahan citra digital menggunakan Python, terdapat tiga library utama yang berperan penting dalam proses pembacaan, pengolahan, dan visualisasi gambar, yaitu OpenCV, NumPy, dan Matplotlib. OpenCV (cv2) digunakan sebagai library inti dalam bidang visi komputer karena menyediakan berbagai fungsi untuk membaca gambar, melakukan transformasi ruang warna, segmentasi objek, filtering, hingga proses deteksi objek pada citra digital. Sementara itu, NumPy (numpy) berfungsi untuk menangani operasi numerik dan pengolahan matriks secara efisien. Hal ini penting karena citra digital pada dasarnya direpresentasikan sebagai array multidimensi yang berisi nilai intensitas piksel. Dengan NumPy, proses manipulasi piksel, pembuatan masker (*masking*), serta penerapan threshold dapat dilakukan dengan lebih cepat dan terstruktur. Adapun Matplotlib (matplotlib.pyplot) digunakan untuk menampilkan hasil pengolahan citra dalam bentuk visual sehingga mudah dianalisis oleh pengguna. Selain menampilkan gambar, library ini juga sering dimanfaatkan untuk membuat histogram yang menggambarkan distribusi intensitas piksel pada citra. Ketiga library tersebut saling melengkapi dan menjadi dasar utama dalam berbagai proses analisis maupun pengolahan citra digital menggunakan Python.

B. Ruang Warna dan Ekstraksi Kanal (Color Segmentation)

- Dalam pengolahan citra digital, pemahaman mengenai ruang warna BGR, RGB, dan HSV sangat penting karena masing-masing format memiliki fungsi dan karakteristik yang berbeda. Secara default, gambar yang dibaca menggunakan fungsi `cv2.imread()` pada OpenCV akan disimpan dalam format BGR (*Blue, Green, Red*), bukan RGB (*Red, Green, Blue*) seperti standar tampilan pada umumnya. Oleh sebab itu, agar warna gambar dapat ditampilkan dengan benar pada library visualisasi seperti Matplotlib, diperlukan proses konversi menggunakan fungsi `cv2.cvtColor()` dengan parameter `cv2.COLOR_BGR2RGB`. Meskipun format RGB cukup baik untuk kebutuhan visualisasi, format ini kurang efektif digunakan dalam proses analisis warna dan segmentasi objek karena sensitif terhadap perubahan pencahayaan. Untuk mengatasi hal tersebut, citra biasanya diubah ke dalam format HSV (*Hue, Saturation, Value*) yang memisahkan informasi warna, tingkat kejenuhan, dan kecerahan. Dengan pemisahan komponen tersebut, proses deteksi dan pemisahan warna pada citra menjadi lebih stabil, akurat, dan mudah dilakukan meskipun kondisi pencahayaan berubah. Format HSV memecah informasi piksel menjadi tiga komponen independen:
 - **Hue (H):** Komponen ini merepresentasikan jenis warna dominan (spektrum warna murni) itu sendiri. Dalam OpenCV, alih-alih menggunakan derajat, nilai Hue dipadatkan ke dalam rentang 0 hingga 179.

- **Saturation (S):** Mengukur tingkat kepekatan atau kemurnian warna, dengan rentang nilai 0 hingga 255. Nilai yang diatur di atas 50 berfungsi sebagai batas bawah agar sistem mengabaikan piksel yang terlalu pudar, keabu-abuan, atau mendekati putih
- **Value (V):** Mewakili kecerahan piksel (0 hingga 255). Batas minimum yang ditetapkan (misalnya 50) sangat penting untuk menghindari deteksi palsu pada piksel yang gelap atau tertutup bayangan hitam
- Teknik *thresholding* dan *masking* digunakan dalam proses ekstraksi warna dengan cara menentukan batas bawah (*lower*) dan batas atas (*upper*) menggunakan array dari NumPy. Pada ruang warna HSV, setiap warna memiliki rentang nilai Hue tertentu yang digunakan sebagai acuan deteksi. Warna biru berada pada rentang Hue 100 hingga 130, sedangkan warna hijau mencakup nilai Hue 35 hingga 85 yang mewakili variasi hijau kekuningan sampai hijau kebiruan. Warna merah memiliki karakteristik khusus karena pada model HSV yang berbentuk silinder, spektrum merah berada di bagian awal dan akhir rentang warna. Oleh sebab itu, deteksi merah memerlukan dua rentang Hue, yaitu 0–10 dan 170–180, agar seluruh variasi merah, termasuk merah keunguan, dapat terdeteksi secara optimal. Proses penyaringan warna dilakukan menggunakan fungsi `cv2.inRange()` yang memindai seluruh piksel pada citra HSV. Piksel yang berada dalam rentang warna yang telah ditentukan akan diubah menjadi putih dengan nilai 255, sedangkan piksel di luar rentang akan menjadi hitam dengan nilai 0 sehingga terbentuk citra biner atau masker. Khusus untuk warna merah, kedua masker hasil deteksi kemudian digabungkan menggunakan fungsi `cv2.bitwise_or()` agar proses segmentasi warna merah menjadi lebih menyeluruh dan akurat. Selanjutnya, hasil masking dapat dievaluasi menggunakan kondisi seperti `mask > 0` untuk menyeleksi area target yang berhasil terdeteksi. Piksel pada area tersebut kemudian dapat diubah menjadi warna putih murni dengan nilai `[255, 255, 255]` sehingga objek terlihat lebih jelas dan menonjol. Selain digunakan secara terpisah, masker warna biru, merah, dan hijau juga dapat digabungkan menggunakan operasi logika OR secara bertahap untuk mendeteksi beberapa objek berwarna sekaligus dalam satu tampilan citra.

C. Operasi Morfologi Citra Biner

Setelah proses *thresholding* dilakukan, masker biner yang dihasilkan umumnya masih mengandung gangguan seperti noise berupa titik-titik hitam di dalam objek putih atau kontur objek yang tidak tersambung dengan baik. Untuk memperbaiki kondisi tersebut, digunakan teknik operasi morfologi yang bertujuan memperhalus dan menyempurnakan hasil segmentasi citra. Dalam operasi ini, komponen utama yang digunakan adalah kernel atau *structuring element*, yaitu matriks kecil yang berfungsi sebagai alat pemrosesan citra. Contohnya adalah matriks berukuran 3×3 yang seluruh elemennya bernilai 1 dan dibuat menggunakan `np.ones((3,3), np.uint8)`. Kernel tersebut bekerja dengan cara bergerak memindai setiap bagian citra biner seperti sebuah stempel kecil yang diterapkan pada seluruh area gambar. Pada proses ini digunakan teknik *Morphological Closing* yang dijalankan melalui

parameter `cv2.MORPH_CLOSE` pada fungsi `cv2.morphologyEx()`. Operasi *Closing* memiliki fungsi utama untuk menutup lubang-lubang kecil dan menyambungkan bagian objek putih yang terputus sehingga bentuk objek menjadi lebih utuh. Dengan penerapan teknik ini, kontur objek terlihat lebih rapi, area segmentasi menjadi lebih terhubung, serta hasil deteksi objek menjadi lebih stabil dan tahan terhadap gangguan visual pada citra digital.

D. Distribusi Intensitas melalui Analisis Histogram

Histogram merupakan salah satu metode analisis yang sangat penting dalam pengolahan citra digital karena mampu menggambarkan distribusi intensitas piksel pada suatu gambar. Fungsi `cv2.calcHist()` digunakan untuk menghitung seluruh nilai piksel pada citra dan memetakan jumlah kemunculannya dalam bentuk grafik histogram. Pada grafik tersebut, sumbu horizontal (X) menunjukkan tingkat intensitas piksel mulai dari 0 yang merepresentasikan warna sangat gelap atau hitam hingga 255 yang menunjukkan warna sangat terang atau putih murni. Sementara itu, sumbu vertikal (Y) menyatakan frekuensi atau jumlah piksel yang memiliki tingkat intensitas tertentu. Suatu citra dikategorikan memiliki distribusi bimodal apabila histogramnya memperlihatkan dua puncak utama yang terpisah cukup jauh. Kondisi ini menunjukkan bahwa citra memiliki tingkat kontras yang tinggi. Puncak pertama pada rentang intensitas rendah, sekitar 10 hingga 40, menandakan dominasi area gelap pada gambar. Sebaliknya, bagian lembah pada kisaran intensitas 50 hingga 120 menunjukkan sangat sedikitnya area abu-abu atau *mid-tone*, sehingga peralihan antara area gelap dan terang terlihat sangat tegas. Puncak kedua pada rentang intensitas sekitar 150 hingga 190 mengindikasikan keberadaan area terang atau *highlight*, seperti objek yang terkena pencahayaan langsung. Karena puncak tersebut tidak mencapai nilai maksimum 255, area terang pada citra masih menyimpan detail visual dan tidak mengalami *overexposure*. Selain itu, analisis terhadap kanal warna merah, hijau, dan biru juga dapat memberikan informasi tambahan. Jika ketiga kurva histogram RGB tidak saling bertumpuk sempurna, maka dapat disimpulkan bahwa citra tersebut merupakan gambar berwarna dan memiliki kecenderungan bias warna tertentu pada tingkat kecerahan yang berbeda, bukan sekadar citra grayscale.

E. Manipulasi Tingkat Kecerahan dan Kontras (Operasi Piksel)

Operasi piksel merupakan proses transformasi yang dilakukan dengan mengubah nilai asli pada citra menjadi nilai baru guna meningkatkan kualitas visual maupun mempermudah sistem dalam mengenali pola tertentu. Sebelum dilakukan manipulasi terhadap rentang dinamis citra, gambar berwarna biasanya terlebih dahulu dikonversi ke dalam format grayscale menggunakan parameter `cv2.COLOR_BGR2GRAY`. Proses konversi ini menghilangkan informasi warna seperti Hue dan Saturation sehingga citra hanya menyisakan data intensitas cahaya atau tingkat kecerahan piksel. Dengan hanya memproses satu lapisan data intensitas, perhitungan matematis pada citra menjadi lebih sederhana, efisien, dan mudah diterapkan pada berbagai teknik pengolahan citra digital seperti peningkatan kontras, deteksi tepi, segmentasi, maupun analisis pola.

Manipulasi dasar dilakukan dengan menerapkan fungsi transformasi linear menggunakan perintah `cv2.convertScaleAbs()`, yang mendasarkan operasinya pada rumus matematika dasar:

$$g(x) = \alpha f(x) + \beta$$

Pada persamaan transformasi citra, $f(x)$ merepresentasikan nilai piksel asli dari gambar sebelum diproses, sedangkan $g(x)$ menunjukkan nilai piksel baru yang dihasilkan setelah proses manipulasi dilakukan. Parameter alpha (α) berfungsi sebagai faktor pengali untuk mengatur tingkat kontras citra. Apabila nilai α lebih besar dari 1, misalnya 2.0, maka perbedaan intensitas antara area gelap dan area terang akan semakin besar sehingga detail visual pada gambar tampak lebih tegas dan kontras. Sementara itu, parameter beta (β) digunakan untuk mengontrol tingkat kecerahan atau brightness dengan cara menambahkan nilai tertentu pada setiap piksel. Ketika nilai β diatur positif, misalnya 60, seluruh intensitas piksel akan bergeser menuju nilai yang lebih tinggi mendekati 255, sehingga gambar terlihat lebih terang secara keseluruhan. Kombinasi kedua parameter tersebut memungkinkan proses penyesuaian kualitas citra dilakukan secara lebih fleksibel sesuai kebutuhan pengolahan dan analisis gambar digital.

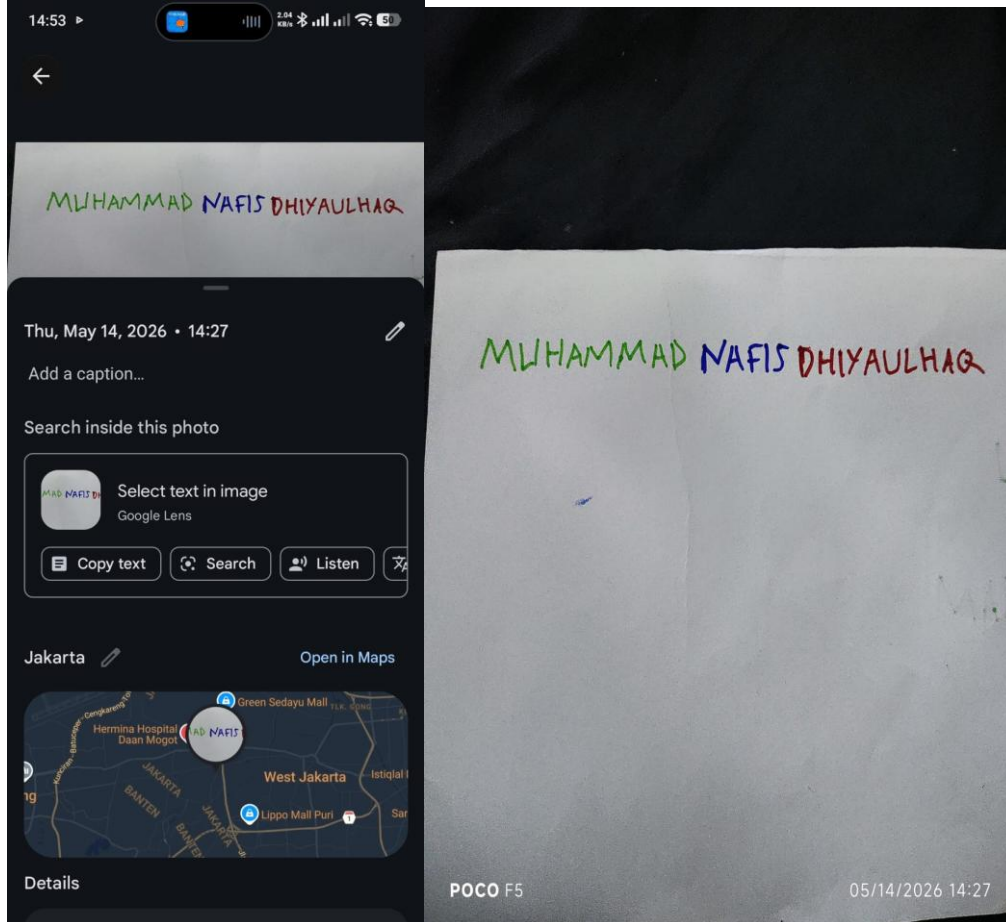
Library OpenCV secara internal memiliki sistem pertahanan yang akan memotong otomatis (clipping) setiap nilai hasil kalkulasi rumusan di atas yang melebihi angka 255 agar kembali menjadi maksimal 255, serta mencegah angka jatuh di bawah 0 (negatif). Kombinasi peningkatan Alpha dan Beta secara bersamaan akan menghasilkan citra yang sangat cerah dengan tepi gradasi yang menonjol tegas.

Semua perbandingan operasi piksel ini divisualisasikan dengan membagi kanvas komputer menjadi sistem grid (kolom dan baris) menggunakan `plt.subplot()`. Sangat penting untuk menyisipkan parameter `cmap='gray'` khusus untuk citra hitam putih agar Matplotlib tidak salah membaca intensitas tunggal menjadi palet warna acak bawaannya.

BAB III

HASIL

1. Ekstraksi kanal warna, ambang batas, dan analisis Histogram



```
In [1]: import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Digunakan untuk mengimpor beberapa library utama yang dibutuhkan dalam proses pengolahan citra digital. Library cv2 yang berasal dari OpenCV berperan dalam membaca, mengolah, serta melakukan berbagai manipulasi pada gambar, seperti deteksi objek, filtering, segmentasi, dan transformasi citra. Selain itu, library numpy dimanfaatkan untuk mendukung operasi numerik dan pengolahan matriks, karena citra digital pada dasarnya direpresentasikan dalam bentuk array multidimensi. Library ini juga sangat membantu dalam proses pembuatan mask, thresholding, serta perhitungan piksel secara efisien.

Sementara itu, matplotlib.pyplot digunakan untuk menampilkan hasil pengolahan citra dalam bentuk visual, baik berupa gambar maupun grafik histogram. Dengan adanya histogram, distribusi intensitas warna atau tingkat kecerahan pada citra dapat dianalisis dengan lebih mudah. Ketiga library tersebut saling melengkapi sehingga proses analisis dan pengolahan citra digital dapat dilakukan secara lebih efektif, terstruktur, dan akurat.


```
In [7]: img = cv2.imread('UTS.jpeg')|
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

Kode tersebut digunakan untuk membaca file gambar bernama UTS.jpeg, kemudian menyimpannya ke dalam variabel `img` dalam bentuk matriks piksel. Setiap piksel pada citra memiliki nilai intensitas warna yang dapat diproses lebih lanjut pada tahapan pengolahan citra digital. Setelah gambar berhasil dibaca, dilakukan proses konversi ruang warna menggunakan fungsi `cv2.cvtColor()` dari library OpenCV. Secara default, OpenCV membaca gambar dalam format BGR (Blue, Green, Red), sehingga perlu diubah ke format HSV (Hue, Saturation, Value) agar proses deteksi dan pemisahan warna dapat dilakukan dengan lebih mudah dan akurat. Format HSV memiliki keunggulan karena memisahkan komponen warna, tingkat kejenuhan, dan tingkat kecerahan, sehingga sangat efektif digunakan dalam proses segmentasi warna seperti merah, hijau, dan biru. Hasil konversi tersebut kemudian disimpan ke dalam variabel baru bernama `img_hsv`, yang berfungsi untuk menampung citra dalam format HSV. Pada fungsi `cv2.cvtColor(img, cv2.COLOR_BGR2HSV)`, parameter pertama berupa `img` merupakan citra asli yang akan diproses, sedangkan parameter kedua `cv2.COLOR_BGR2HSV` merupakan instruksi untuk mengubah ruang warna dari BGR menjadi HSV. Dengan konversi ini, analisis warna pada citra menjadi lebih stabil dan efisien, terutama ketika digunakan pada proses thresholding, masking, maupun deteksi objek berdasarkan warna tertentu.

```
In [8]: lower_blue, upper_blue = np.array([100, 50, 50]), np.array([130, 255, 255])
lower_red1, upper_red1 = np.array([0, 50, 50]), np.array([10, 255, 255])
lower_red2, upper_red2 = np.array([170, 50, 50]), np.array([180, 255, 255])
lower_green, upper_green = np.array([35, 50, 50]), np.array([85, 255, 255])
```

Array [H, S, V] pada format HSV digunakan untuk merepresentasikan komponen warna, tingkat kejenuhan, dan tingkat kecerahan suatu citra. Pada contoh nilai [100, 50, 50], angka pertama menunjukkan Hue (H), yaitu spektrum warna dengan rentang nilai 0–179 pada OpenCV. Angka kedua merupakan Saturation (S) yang menunjukkan tingkat kepekatan warna dengan rentang 0–255. Nilai batas bawah sebesar 50 digunakan agar sistem dapat mengabaikan warna yang terlalu pudar, keabu-abuan, atau mendekati putih, sehingga warna yang terdeteksi hanya warna yang memiliki intensitas cukup jelas. Angka ketiga adalah Value (V), yaitu tingkat kecerahan dengan rentang 0–255. Penetapan nilai minimum 50 bertujuan untuk menghindari deteksi pada piksel yang terlalu gelap atau hitam, misalnya akibat bayangan pada gambar. Dalam proses segmentasi warna, setiap warna memiliki rentang Hue tertentu. Warna biru, misalnya, berada pada rentang Hue 100 hingga 130 yang secara khusus merepresentasikan spektrum biru pada HSV di OpenCV. Sementara itu, warna merah memerlukan dua rentang nilai, yaitu Hue 0–10 dan 170–180. Hal ini disebabkan model HSV berbentuk lingkaran sehingga warna merah terletak pada bagian awal dan akhir spektrum warna. Oleh karena itu, dua rentang diperlukan agar seluruh variasi warna merah dapat terdeteksi secara optimal. Adapun warna hijau berada pada rentang Hue 35 hingga 85 yang mencakup spektrum hijau kekuningan hingga hijau kebiruan. Pengaturan rentang HSV tersebut sangat penting untuk meningkatkan akurasi deteksi dan pemisahan objek berdasarkan warna pada proses pengolahan citra digital.

```
In [12]: mask_b = cv2.inRange(img_hsv, lower_blue, upper_blue)
mask_g = cv2.inRange(img_hsv, lower_green, upper_green)
mask_r = cv2.bitwise_or(cv2.inRange(img_hsv, lower_red1, upper_red1), cv2.inRange(img_hsv, lower_red2, upper_red2))

res_b = cv2.bitwise_or(img_rgb, img_rgb, mask=cv2.bitwise_not(mask_b))
res_r = cv2.bitwise_or(img_rgb, img_rgb, mask=cv2.bitwise_not(mask_r))
res_g = cv2.bitwise_or(img_rgb, img_rgb, mask=cv2.bitwise_not(mask_g))
```

fungsi `cv2.inRange()` digunakan untuk memindai seluruh citra dan menghasilkan gambar biner berupa warna hitam dan putih. Pada `mask_b = cv2.inRange(...)`, sistem akan mendeteksi piksel pada `img_hsv` yang berada dalam rentang warna biru. Piksel yang sesuai dengan kriteria warna biru akan diberi nilai putih (255), sedangkan piksel lainnya akan diubah menjadi hitam (0). Proses yang sama juga diterapkan pada `mask_g = cv2.inRange(...)` untuk mendeteksi warna hijau. Sementara itu, karena warna merah pada model HSV berada di dua bagian spektrum warna, maka dibuat dua masker merah terlebih dahulu. Kedua masker tersebut kemudian digabungkan menggunakan fungsi `cv2.bitwise_or()` sehingga menghasilkan satu masker merah yang utuh dan lebih akurat. Setelah masker selesai dibuat, langkah berikutnya adalah menerapkan masker tersebut pada gambar asli. Pada baris `res_b = cv2.bitwise_or(img_rgb, img_rgb, mask=cv2.bitwise_not(mask_b))`, fungsi `cv2.bitwise_not(mask_b)` digunakan untuk membalik nilai masker, sehingga area warna biru yang sebelumnya berwarna putih berubah menjadi hitam, sedangkan bagian latar belakang yang hitam berubah menjadi putih. Selanjutnya, fungsi `cv2.bitwise_or()` digunakan untuk menggabungkan gambar asli dengan dirinya sendiri berdasarkan area masker yang bernilai putih. Dengan metode ini, bagian tertentu dari citra dapat dipertahankan atau dihilangkan sesuai kebutuhan proses segmentasi warna. Teknik masking seperti ini sangat penting dalam pengolahan citra digital karena memungkinkan sistem melakukan pemisahan objek berdasarkan warna secara lebih efektif dan terstruktur.

```
In [13]: res_b[mask_b > 0] = [255, 255, 255]
res_r[mask_r > 0] = [255, 255, 255]
res_g[mask_g > 0] = [255, 255, 255]
```

Ekspresi `mask_b > 0` digunakan untuk mengevaluasi citra masker biner yang sebelumnya telah dibuat. Pada masker biner, umumnya hanya terdapat dua nilai, yaitu 0 untuk area yang tidak termasuk target dan 255 untuk area yang berhasil mendeteksi warna tertentu. Kondisi `> 0` berfungsi menyeleksi seluruh koordinat piksel yang teridentifikasi sebagai warna target, dalam hal ini area dengan warna biru. Selanjutnya, penulisan `res_b[...]` digunakan untuk mengakses citra hasil pengolahan (`res_b`). Dengan memasukkan kondisi masker ke dalam tanda kurung siku, program akan memilih piksel pada `res_b` yang posisinya sesuai dengan area target pada `mask_b`. Setelah piksel target berhasil dipilih, perintah `= [255, 255, 255]` digunakan untuk mengubah nilai warna piksel tersebut menjadi putih murni dalam format RGB maupun BGR. Proses ini diterapkan dengan logika yang sama pada warna lainnya, seperti merah dan hijau, melalui variabel `res_r` dan `res_g`. Secara keseluruhan, kode tersebut bekerja dengan cara mendeteksi posisi tulisan berdasarkan warna tertentu, kemudian mengganti warna asli tulisan tersebut menjadi putih solid. Teknik ini berguna dalam proses segmentasi dan penonjolan objek pada pengolahan citra digital karena dapat memperjelas area target yang ingin dianalisis lebih lanjut.

```

In [32]: plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(img_rgb)
plt.title('Citra Kontras')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(res_b)
plt.title('Biru')
plt.axis('on')

plt.subplot(2, 2, 3)
plt.imshow(res_r)
plt.title('Merah')
plt.axis('on')

plt.subplot(2, 2, 4)
plt.imshow(res_g)
plt.title('Hijau')
plt.axis('on')

plt.tight_layout()
plt.show()

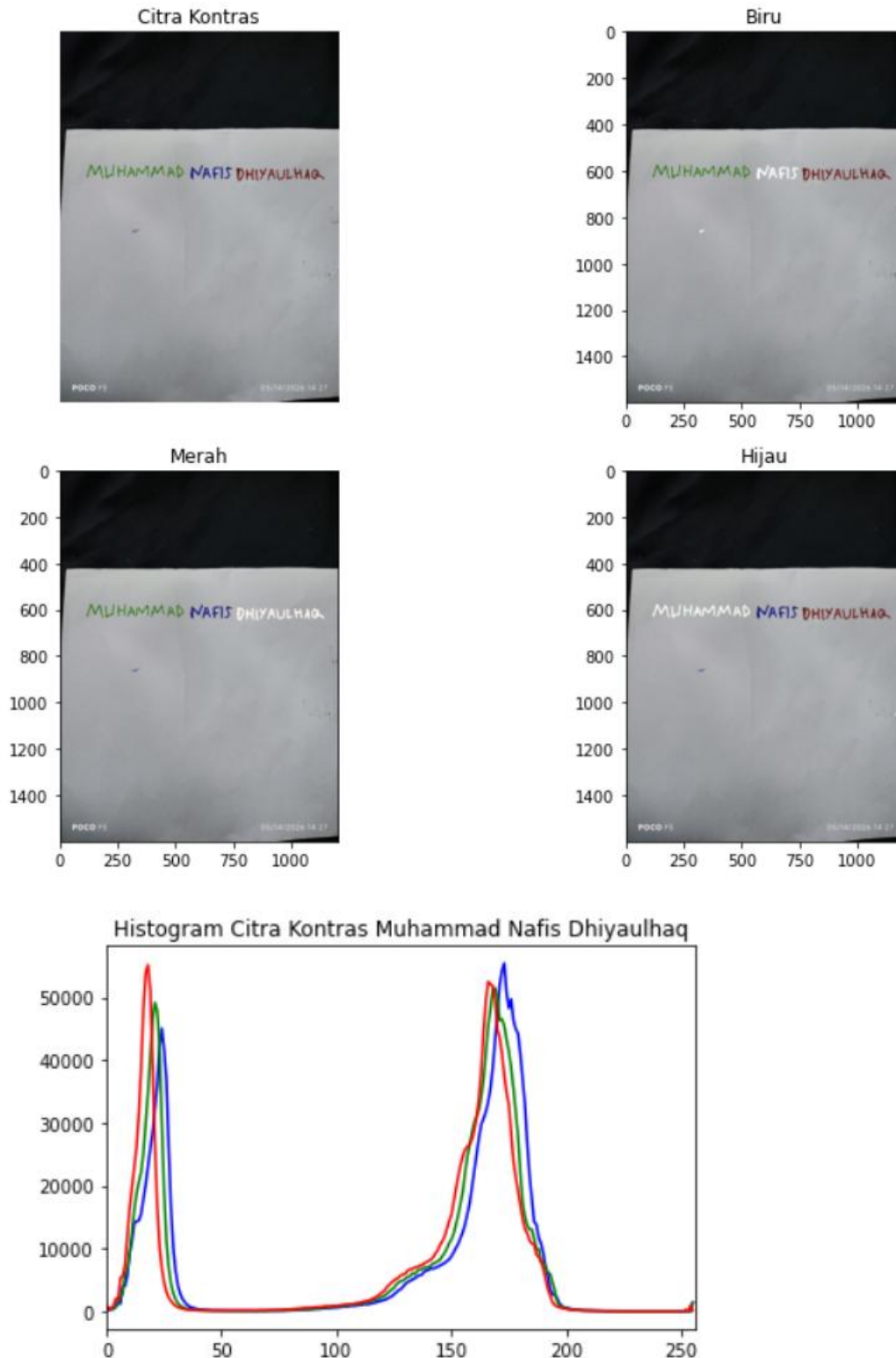
plt.figure(figsize=(6, 4))
colors = ('b', 'g', 'r')
for i, col in enumerate(colors):
    hist = cv2.calcHist([img], [i], None, [256], [0, 256])
    plt.plot(hist, color=col)
plt.title('Histogram Citra Kontras Muhammad Nafis Dhiyaulhaq')
plt.xlim([0, 256])
plt.show()

```

kode bertujuan untuk menampilkan empat gambar dalam satu tampilan menggunakan konsep subplot pada Matplotlib. Proses diawali dengan `plt.figure(figsize=(12, 8))` yang digunakan untuk membuat kanvas atau area tampilan baru dengan ukuran lebar 12 inci dan tinggi 8 inci. Selanjutnya, fungsi `plt.subplot(2, 2, 1)` membagi kanvas menjadi grid berukuran 2 baris dan 2 kolom, kemudian menempatkan gambar pertama pada posisi kiri atas. Fungsi `plt.imshow()` digunakan untuk menampilkan citra pada subplot yang aktif, sedangkan `plt.title()` berfungsi memberikan judul pada setiap gambar agar lebih mudah dikenali. Pengaturan `plt.axis('off')` atau `plt.axis('on')` digunakan untuk menyembunyikan maupun menampilkan sumbu koordinat piksel pada gambar. Pada implementasinya, gambar asli ditampilkan tanpa sumbu, sementara hasil ekstraksi warna tetap menampilkan sumbu koordinat. Pola subplot tersebut diulangi untuk menampilkan citra hasil segmentasi warna biru, merah, dan hijau pada posisi yang berbeda dalam grid. Agar tampilan antar subplot lebih rapi dan tidak saling bertumpuk, digunakan fungsi `plt.tight_layout()`.

Setelah seluruh gambar selesai disusun, `plt.show()` dijalankan untuk menampilkan hasil akhir ke layar. Selain menampilkan gambar, kode juga digunakan untuk membuat histogram RGB yang berfungsi memperlihatkan distribusi intensitas piksel pada citra. Proses ini dimulai dengan `plt.figure(figsize=(6, 4))` untuk membuat kanvas baru khusus histogram. Variabel `colors = ('b', 'g', 'r')` digunakan untuk menentukan warna garis histogram, yaitu biru, hijau, dan merah. Selanjutnya, dilakukan perulangan menggunakan `for i, col in enumerate(colors):` untuk menghitung histogram setiap kanal warna secara bergantian. Fungsi `cv2.calcHist([img], [i], None, [256], [0, 256])` digunakan untuk menghitung jumlah piksel berdasarkan intensitas warna pada masing-masing kanal. Parameter `[img]` menunjukkan gambar sumber, `[i]` menentukan kanal warna yang diproses, `None` menandakan tidak adanya masker, `[256]` menyatakan jumlah bin histogram, dan `[0, 256]` menunjukkan rentang intensitas piksel dari 0 hingga 255. Hasil histogram kemudian divisualisasikan menggunakan `plt.plot(hist, color=col)`.

dengan warna garis sesuai kanal yang dihitung. Grafik juga diberi judul menggunakan `plt.title()`, sedangkan `plt.xlim([0, 256])` digunakan untuk membatasi tampilan sumbu X agar tetap berada pada rentang intensitas piksel yang valid. Terakhir, `plt.show()` digunakan untuk menampilkan grafik histogram ke layar sehingga distribusi warna pada citra dapat dianalisis secara visual dengan lebih mudah.



Pada histogram citra, sumbu horizontal (X) menunjukkan nilai intensitas piksel dengan rentang 0 hingga 255. Nilai yang mendekati 0 merepresentasikan area yang sangat gelap atau hitam, sedangkan nilai yang mendekati 255 menunjukkan area yang sangat terang

atau putih. Sementara itu, sumbu vertikal (Y) menunjukkan frekuensi atau jumlah piksel yang memiliki tingkat intensitas tertentu. Semakin tinggi kurva pada suatu titik, semakin banyak piksel pada gambar yang berada pada tingkat kecerahan tersebut. Berdasarkan bentuk grafiknya, histogram ini termasuk dalam distribusi bimodal karena memiliki dua puncak utama yang terpisah cukup jauh. Karakteristik ini menunjukkan bahwa citra memiliki tingkat kontras yang tinggi. Puncak pertama berada pada rentang intensitas rendah, sekitar 10 hingga 40, yang menandakan dominasi area gelap dalam gambar. Di bagian tengah histogram, yaitu pada kisaran intensitas sekitar 50 hingga 120, grafik terlihat sangat rendah dan hampir mendekati nol. Hal tersebut menunjukkan bahwa citra memiliki sangat sedikit area dengan warna abu-abu atau tingkat kecerahan menengah, sehingga perpindahan antara area gelap dan terang terlihat sangat tegas. Puncak kedua muncul pada rentang intensitas sekitar 150 hingga 190 yang menunjukkan keberadaan area terang atau highlight pada citra, seperti latar belakang terang atau objek yang terkena pencahayaan. Karena puncaknya tidak berada tepat di nilai maksimum 255, area terang tersebut masih memiliki detail dan tidak mengalami overexposure. Histogram juga menampilkan tiga kurva warna yang mewakili kanal merah (Red), hijau (Green), dan biru (Blue). Ketiga garis tersebut tidak bertumpuk secara sempurna, yang menandakan bahwa gambar merupakan citra berwarna, bukan grayscale. Perbedaan posisi antar garis menunjukkan adanya kecenderungan atau bias warna tertentu pada area gelap maupun terang, misalnya dominasi rona kemerahan atau kebiruan pada bagian tertentu dari citra.

```
In [26]: lower_blue = np.array([100, 50, 50])
         upper_blue = np.array([130, 255, 255])

         lower_red1 = np.array([0, 50, 50])
         upper_red1 = np.array([10, 255, 255])
         lower_red2 = np.array([170, 50, 50])
         upper_red2 = np.array([180, 255, 255])

         lower_green = np.array([35, 50, 50])
         upper_green = np.array([85, 255, 255])
```

Fungsi `np.array(...)` digunakan untuk menyimpan tiga komponen utama pada ruang warna HSV, yaitu Hue (H), Saturation (S), dan Value (V), yang berfungsi sebagai batas deteksi warna pada citra. Pada seluruh rentang warna, nilai Saturation dan Value ditetapkan dari 50 hingga 255. Penentuan batas bawah di angka 50 bertujuan untuk mengurangi gangguan dari piksel yang terlalu pucat, terlalu putih, maupun terlalu gelap akibat bayangan, sehingga sistem hanya mendeteksi warna yang memiliki tingkat kepekatan dan kecerahan yang cukup jelas, seperti tinta spidol atau pena pada permukaan kertas putih. Sementara itu, komponen Hue digunakan untuk menentukan jenis warna yang akan dideteksi. Untuk warna biru, rentang Hue yang digunakan adalah 100 hingga 130 karena area tersebut merepresentasikan spektrum warna biru pada format HSV di OpenCV, sehingga pemisahan warna dapat dilakukan dengan lebih akurat. Pada warna hijau, digunakan rentang Hue 35 hingga 85 agar berbagai variasi warna hijau, mulai dari hijau kekuningan hingga hijau daun, tetap dapat terdeteksi dengan baik. Berbeda dengan warna lainnya, warna merah memerlukan dua rentang Hue, yaitu 0–10 dan 170–180. Hal ini disebabkan model HSV berbentuk silinder

sehingga warna merah berada pada titik awal dan akhir spektrum warna. Jika hanya menggunakan satu rentang, beberapa variasi merah yang cenderung keunguan tidak akan terdeteksi secara optimal. Oleh karena itu, kedua rentang tersebut digunakan dan kemudian digabungkan menggunakan operasi bitwise_or agar proses deteksi warna merah menjadi lebih merata, akurat, dan mampu mencakup seluruh spektrum merah pada citra digital.

```
In [27]: mask_b = cv2.inRange(img_hsv, lower_blue, upper_blue)

        mask_r1 = cv2.inRange(img_hsv, lower_red1, upper_red1)
        mask_r2 = cv2.inRange(img_hsv, lower_red2, upper_red2)

        mask_r = cv2.bitwise_or(mask_r1, mask_r2)
        |
        mask_g = cv2.inRange(img_hsv, lower_green, upper_green)
```

Fungsi `cv2.inRange()` bekerja sebagai mekanisme penyaringan piksel pada citra HSV dengan cara memeriksa setiap piksel berdasarkan rentang warna yang telah ditentukan. Jika nilai warna suatu piksel berada di antara batas bawah (lower) dan batas atas (upper), maka piksel tersebut akan diubah menjadi putih dengan nilai 255. Sebaliknya, apabila berada di luar rentang yang ditentukan, piksel akan diubah menjadi hitam dengan nilai 0. Pada baris `mask_b = cv2.inRange(img_hsv, lower_blue, upper_blue)`, fungsi ini digunakan untuk membuat masker warna biru sehingga hanya area yang mengandung warna biru yang tampil putih, sedangkan latar belakang dan warna lainnya menjadi hitam. Untuk warna merah, prosesnya sedikit berbeda karena spektrum merah pada HSV berada di dua sisi rentang warna, yaitu di dekat nilai 0 dan 179. Oleh sebab itu, dibuat dua masker terpisah melalui `mask_r1` dan `mask_r2`. Masker pertama digunakan untuk mendeteksi merah yang mendekati oranye, sedangkan masker kedua mendeteksi merah yang mendekati ungu atau magenta. Setelah kedua masker diperoleh, fungsi `cv2.bitwise_or()` digunakan untuk menggabungkannya. Operasi logika OR tersebut akan menghasilkan piksel putih apabila piksel pada salah satu masker bernilai putih, sehingga terbentuk satu masker merah yang mampu mendeteksi seluruh spektrum warna merah secara lebih lengkap dan akurat. Sementara itu, `mask_g = cv2.inRange(img_hsv, lower_green, upper_green)` digunakan untuk melakukan proses penyaringan warna hijau dengan prinsip kerja yang sama seperti pada warna biru. Teknik masking ini sangat penting dalam pengolahan citra digital karena memungkinkan pemisahan objek berdasarkan warna tertentu secara lebih efektif dan terstruktur.

```
In [28]: kernel = np.ones((3,3), np.uint8)

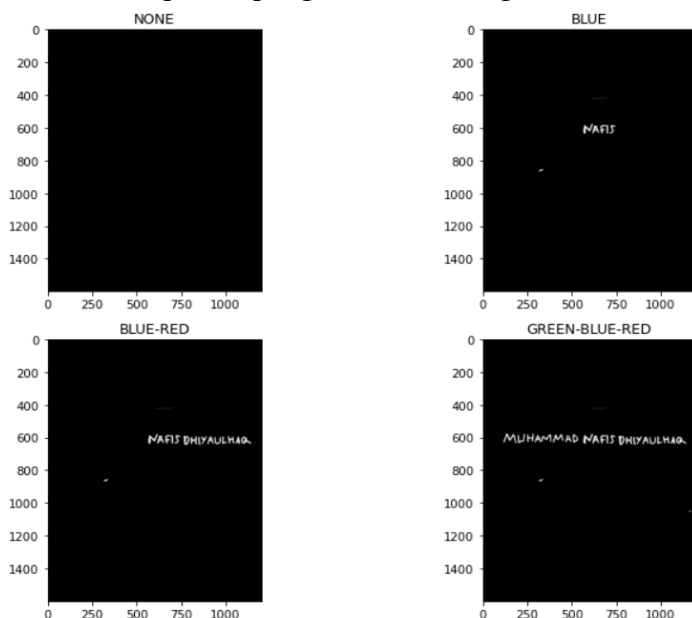
        mask_b = cv2.morphologyEx(mask_b, cv2.MORPH_CLOSE, kernel)
        mask_r = cv2.morphologyEx(mask_r, cv2.MORPH_CLOSE, kernel)
        mask_g = cv2.morphologyEx(mask_g, cv2.MORPH_CLOSE, kernel)|
```

Kernel atau elemen penstruktur (structuring element) merupakan matriks berukuran kecil yang digunakan dalam operasi morfologi untuk memindai setiap bagian citra secara bertahap. Pada kode `kernel = np.ones((3,3), np.uint8)`, dibuat sebuah matriks berukuran 3×3 yang seluruh elemennya bernilai 1 dengan tipe data `uint8`. Kernel ini dapat dianalogikan sebagai sebuah kuas atau stempel kecil berbentuk persegi yang digunakan untuk memproses area tertentu pada citra, khususnya pada bagian masker biner. Setelah kernel dibuat, langkah berikutnya adalah menerapkan operasi morfologi menggunakan fungsi `cv2.morphologyEx()`,

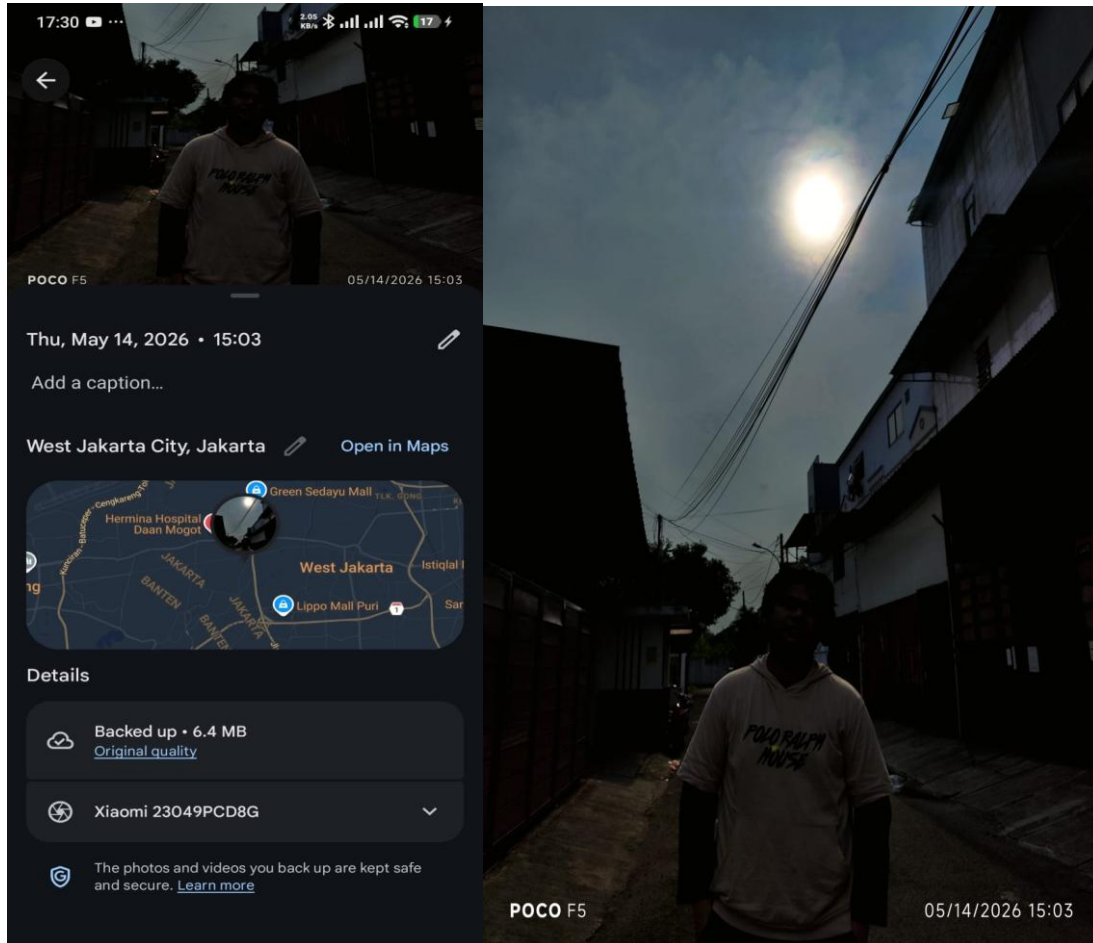
yaitu fungsi bawaan OpenCV yang digunakan untuk menjalankan berbagai teknik pemrosesan morfologi tingkat lanjut. Pada kode tersebut digunakan parameter `cv2.MORPH_CLOSE`, yang menandakan bahwa operasi yang diterapkan adalah teknik *Closing*. Operasi ini bertujuan untuk menutup celah kecil atau lubang pada area objek berwarna putih dalam masker, sekaligus merapikan bentuk objek agar hasil segmentasi menjadi lebih halus dan bersih. Dengan penggunaan kernel dan teknik *Closing*, noise kecil pada citra dapat dikurangi sehingga proses deteksi objek berdasarkan warna menjadi lebih akurat dan stabil.

```
In [29]: mask_none = np.zeros_like(mask_b)
         mask_blue = mask_b
         mask_blue_red = cv2.bitwise_or(mask_b, mask_r)
         mask_all = cv2.bitwise_or(mask_blue_red, mask_g)
```

Kode `mask_none = np.zeros_like(mask_b)` digunakan untuk membuat sebuah matriks baru yang seluruh nilainya berisi angka 0 dengan ukuran yang sama seperti `mask_b`. Fungsi `np.zeros_like()` dari library NumPy bekerja dengan meniru dimensi array referensi, sehingga hasilnya berupa citra kosong berwarna hitam karena nilai 0 pada citra digital merepresentasikan warna hitam. Matriks ini biasanya digunakan sebagai tampilan dasar atau kondisi “NONE” tanpa adanya objek yang terdeteksi. Selanjutnya, pada `mask_blue = mask_b`, data dari masker biru disalin ke variabel baru bernama `mask_blue`, yang berfungsi untuk menampilkan hasil segmentasi warna biru secara terpisah. Proses berikutnya dilakukan melalui `mask_blue_red = cv2.bitwise_or(mask_b, mask_r)`, yaitu operasi logika OR yang digunakan untuk menggabungkan dua masker berbeda. Dalam proses ini, piksel akan bernilai putih apabila pada masker biru atau masker merah terdapat piksel putih pada posisi yang sama. Hasilnya adalah sebuah masker gabungan yang menampilkan area warna biru dan merah secara bersamaan. Setelah itu, proses penggabungan dilanjutkan pada `mask_all = cv2.bitwise_or(mask_blue_red, mask_g)`, yaitu dengan mengombinasikan masker gabungan biru-merah dengan masker hijau. Operasi ini menghasilkan satu masker lengkap yang mampu mendeteksi seluruh warna target, yaitu biru, merah, dan hijau. Dengan teknik penggabungan tersebut, sistem dapat menampilkan hasil segmentasi beberapa warna sekaligus secara lebih efektif dalam proses pengolahan citra digital.



2. Operasi Pixel



```
In [33]: img = cv2.imread('OPEX.jpeg')
         img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

Kode `img = cv2.imread('OPEX.jpeg')` digunakan untuk membaca atau memuat file gambar bernama `OPEX.jpeg` dari penyimpanan komputer ke dalam program Python menggunakan fungsi `cv2.imread()` dari library OpenCV. Agar proses pembacaan berhasil, file gambar tersebut harus berada pada direktori yang sama dengan script Python yang dijalankan atau disertai dengan lokasi path yang sesuai. Hasil pembacaan gambar kemudian disimpan ke dalam variabel `img` dalam bentuk matriks piksel yang berisi informasi nilai warna pada setiap piksel citra. Secara bawaan, OpenCV menyimpan urutan warna gambar dalam format BGR (Blue, Green, Red), bukan RGB (Red, Green, Blue) seperti format warna yang umum digunakan pada berbagai aplikasi visualisasi. Oleh karena itu, dilakukan proses konversi warna menggunakan kode `img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`. Fungsi `cv2.cvtColor()` digunakan untuk mengubah ruang atau format warna citra, sedangkan parameter `cv2.COLOR_BGR2RGB` berfungsi sebagai instruksi untuk menukar urutan kanal warna dari BGR menjadi RGB. Hasil konversi tersebut kemudian disimpan ke dalam variabel `img_rgb`, sehingga gambar dapat ditampilkan dengan komposisi warna yang sesuai dan lebih akurat pada proses visualisasi maupun analisis citra digital.


```
In [36]: gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

        bright = cv2.convertScaleAbs(gray, alpha=1.0, beta=60)
        contrast = cv2.convertScaleAbs(gray, alpha=2.0, beta=0)
        bright_contrast = cv2.convertScaleAbs(gray, alpha=2.0, beta=60)
```

Kode `gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)` digunakan untuk mengubah gambar asli yang masih berada dalam format BGR menjadi citra grayscale atau hitam-putih. Pada proses ini, informasi warna seperti Hue dan Saturation dihilangkan sehingga hanya tersisa informasi intensitas cahaya atau tingkat kecerahan piksel. Konversi ke grayscale merupakan tahap penting sebelum dilakukan manipulasi terhadap tingkat kecerahan dan kontras citra. Selanjutnya, fungsi `cv2.convertScaleAbs()` digunakan untuk melakukan transformasi linear pada setiap piksel gambar berdasarkan persamaan berikut: $g(x) = \alpha f(x) + \beta$. Pada persamaan tersebut, $f(x)$ menyatakan nilai piksel asli, sedangkan $g(x)$ merupakan nilai piksel hasil transformasi. Parameter α digunakan untuk mengatur kontras citra, di mana nilai lebih besar dari 1 akan membuat perbedaan antara area gelap dan terang menjadi semakin jelas. Sementara itu, parameter β berfungsi untuk mengontrol tingkat kecerahan dengan cara menambahkan nilai tertentu pada setiap piksel. Fungsi ini juga memiliki mekanisme pembatasan otomatis sehingga nilai piksel yang melebihi 255 akan dipotong menjadi 255 dan tidak menghasilkan nilai negatif. Pada kode `bright = cv2.convertScaleAbs(gray, alpha=1.0, beta=60)`, nilai kontras dibiarkan tetap karena $\alpha = 1.0$, sedangkan kecerahan ditingkatkan dengan menambahkan nilai 60 pada setiap piksel sehingga gambar tampak lebih terang. Kemudian, pada `contrast = cv2.convertScaleAbs(gray, alpha=2.0, beta=0)`, nilai kontras ditingkatkan dengan mengalikan setiap piksel sebesar 2 tanpa menambah kecerahan dasar, sehingga perbedaan antara area terang dan gelap menjadi lebih tajam. Adapun pada `bright_contrast = cv2.convertScaleAbs(gray, alpha=2.0, beta=60)`, kedua proses tersebut digabungkan sekaligus, yaitu meningkatkan kontras dan menambahkan kecerahan, sehingga citra menjadi lebih terang dengan detail perbedaan intensitas yang lebih jelas.

```
In [37]: plt.figure(figsize=(14,8))

        plt.subplot(2,3,1)
        plt.imshow(img_rgb)
        plt.title('Gambar Asli')
        plt.axis('off')

        plt.subplot(2,3,2)
        plt.imshow(gray, cmap='gray')
        plt.title('Grayscale')
        plt.axis('off')

        plt.subplot(2,3,3)
        plt.imshow(bright, cmap='gray')
        plt.title('Brightness')
        plt.axis('off')

        plt.subplot(2,3,4)
        plt.imshow(contrast, cmap='gray')
        plt.title('Contrast')
        plt.axis('off')

        plt.subplot(2,3,5)
        plt.imshow(bright_contrast, cmap='gray')
        plt.title('Brightness + Contrast')
        plt.axis('off')

        plt.tight_layout()
        plt.show()
```

Kode `plt.figure(figsize=(14,8))` digunakan untuk membuat sebuah kanvas atau area tampilan baru dengan ukuran lebar 14 inci dan tinggi 8 inci sehingga seluruh gambar dapat ditampilkan dengan lebih jelas dan tidak saling berhimpitan. Setelah kanvas dibuat, fungsi `plt.subplot(2, 3, x)` digunakan untuk membagi area tampilan menjadi grid yang terdiri atas 2 baris dan 3 kolom, sehingga tersedia enam slot untuk penempatan gambar. Parameter ketiga pada fungsi tersebut menentukan posisi gambar dalam grid, yang dihitung dari kiri ke kanan lalu berlanjut ke baris berikutnya. Pada setiap subplot, fungsi `plt.imshow()` digunakan untuk menampilkan citra. Untuk gambar asli berformat RGB, citra dapat ditampilkan secara langsung menggunakan `plt.imshow(img_rgb)`. Sementara itu, pada citra grayscale atau hasil manipulasi piksel, digunakan parameter tambahan `cmap='gray'` agar gambar hitam-putih ditampilkan dengan skala abu-abu yang sesuai dan tidak muncul dalam warna palsu akibat pengaturan bawaan Matplotlib. Setiap gambar kemudian diberi judul menggunakan `plt.title()` untuk menjelaskan jenis citra atau efek yang diterapkan, seperti gambar asli, peningkatan brightness, maupun kombinasi brightness dan contrast. Selain itu, `plt.axis('off')` digunakan untuk menyembunyikan garis tepi dan koordinat sumbu X serta Y sehingga tampilan gambar menjadi lebih bersih dan fokus pada objek citra. Setelah seluruh gambar selesai diatur, fungsi `plt.tight_layout()` digunakan untuk merapikan jarak antar subplot agar judul dan gambar tidak saling bertumpuk. Terakhir, `plt.show()` dijalankan untuk menampilkan seluruh hasil visualisasi ke layar komputer sehingga proses perbandingan antar citra dapat diamati dengan lebih mudah dan terstruktur.



BAB IV

PENUTUP

A. Kesimpulan

Penggunaan library OpenCV, NumPy, dan Matplotlib dalam pemrograman Python menjadi dasar penting dalam proses pengolahan dan analisis citra digital. OpenCV digunakan untuk menangani berbagai proses teknis seperti konversi ruang warna, filtering, segmentasi, dan manipulasi citra. Sementara itu, NumPy berperan dalam mendukung operasi numerik pada matriks piksel karena citra digital direpresentasikan dalam bentuk array multidimensi. Adapun Matplotlib digunakan untuk menampilkan hasil pengolahan citra serta memvisualisasikan distribusi data piksel melalui grafik histogram. Dalam proses segmentasi warna, konversi citra dari format BGR ke HSV (Hue, Saturation, Value) terbukti lebih efektif dibandingkan RGB karena HSV memisahkan informasi warna, tingkat kejenuhan, dan tingkat kecerahan sehingga deteksi warna menjadi lebih stabil terhadap perubahan pencahayaan. Proses ekstraksi warna menggunakan fungsi `cv2.inRange()` mampu memisahkan objek berwarna seperti biru, hijau, dan merah dalam bentuk masker biner. Khusus untuk warna merah, diperlukan dua rentang nilai Hue yang kemudian digabungkan menggunakan operasi `bitwise_or` karena spektrum merah berada pada bagian awal dan akhir lingkaran HSV. Selain itu, penerapan operasi morfologi Closing menggunakan kernel matriks berukuran 3×3 terbukti efektif dalam mengurangi noise dengan cara menutup celah kecil pada area objek sehingga hasil segmentasi menjadi lebih rapi dan solid.

Melalui analisis histogram, dapat diketahui bahwa citra dengan distribusi bimodal memiliki tingkat kontras yang tinggi karena adanya perbedaan yang jelas antara area gelap dan area terang. Histogram memberikan informasi mengenai distribusi frekuensi piksel pada setiap tingkat intensitas dari 0 hingga 255, sehingga kualitas pencahayaan dan kontras citra dapat dianalisis secara objektif. Selain itu, manipulasi citra pada tingkat piksel menggunakan transformasi linear $g(x) = \alpha f(x) + \beta$ mampu meningkatkan kualitas visual gambar secara signifikan. Parameter alpha (α) digunakan untuk mengatur tingkat kontras sehingga detail gambar terlihat lebih tegas, sedangkan parameter beta (β) berfungsi untuk mengontrol tingkat kecerahan atau brightness secara keseluruhan. Kombinasi kedua parameter tersebut memungkinkan proses penyesuaian kualitas citra dilakukan secara lebih fleksibel sesuai kebutuhan analisis maupun visualisasi.

DAFTAR PUSTAKA

Pradana, A. W., & Kusumanto, R. D. (2025). Analisis Kinerja Segmentasi Warna Menggunakan Ruang HSV pada Deteksi Objek Real-Time. Jurnal Informatika dan Rekayasa Elektronik, 8(1), 112-120.

Hidayatullah, P. (2024). Penerapan Operasi Morfologi Closing untuk Reduksi Noise pada Citra Biner. Jurnal Sistem Komputer dan Kecerdasan Buatan, 12(2), 45-53

Setyawan, I., & Nugroho, H. A. (2023). Transformasi Linear Citra Digital: Evaluasi Parameter Alpha dan Beta pada Peningkatan Kualitas Citra Medis Grayscale. Jurnal Teknologi Informasi dan Ilmu Komputer, 10(4), 301-308.

Lestari, D. P., & Syukur, A. (2022). Identifikasi Distribusi Bimodal pada Citra Kontras Tinggi Menggunakan Analisis Histogram OpenCV. Jurnal Komputer Terapan, 7(3), 210-218.

Wibowo, S. A., dkk. (2021). Implementasi Library Python (OpenCV, NumPy, Matplotlib) dalam Rancang Bangun Sistem Computer Vision Otomatis. Jurnal Rekayasa Perangkat Lunak, 5(1), 88-95.